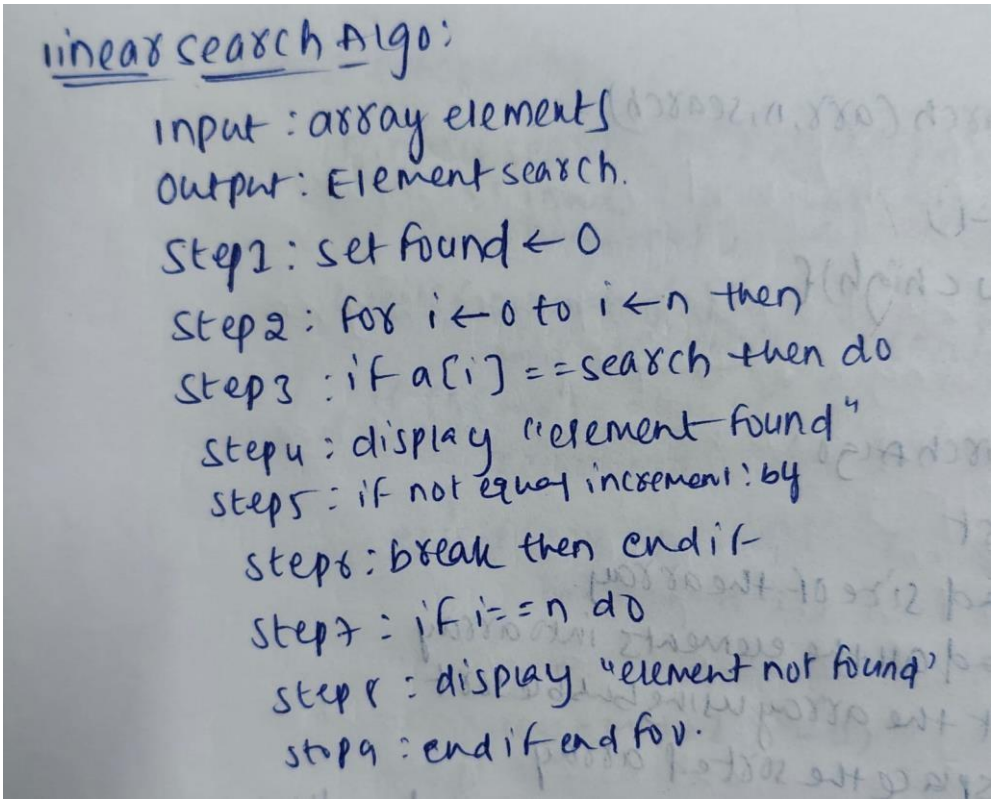
Practical: 2	Implementation and Time analysis of linear and binary search algorithm
05/08/2026	

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

Linear search

Algorithm:



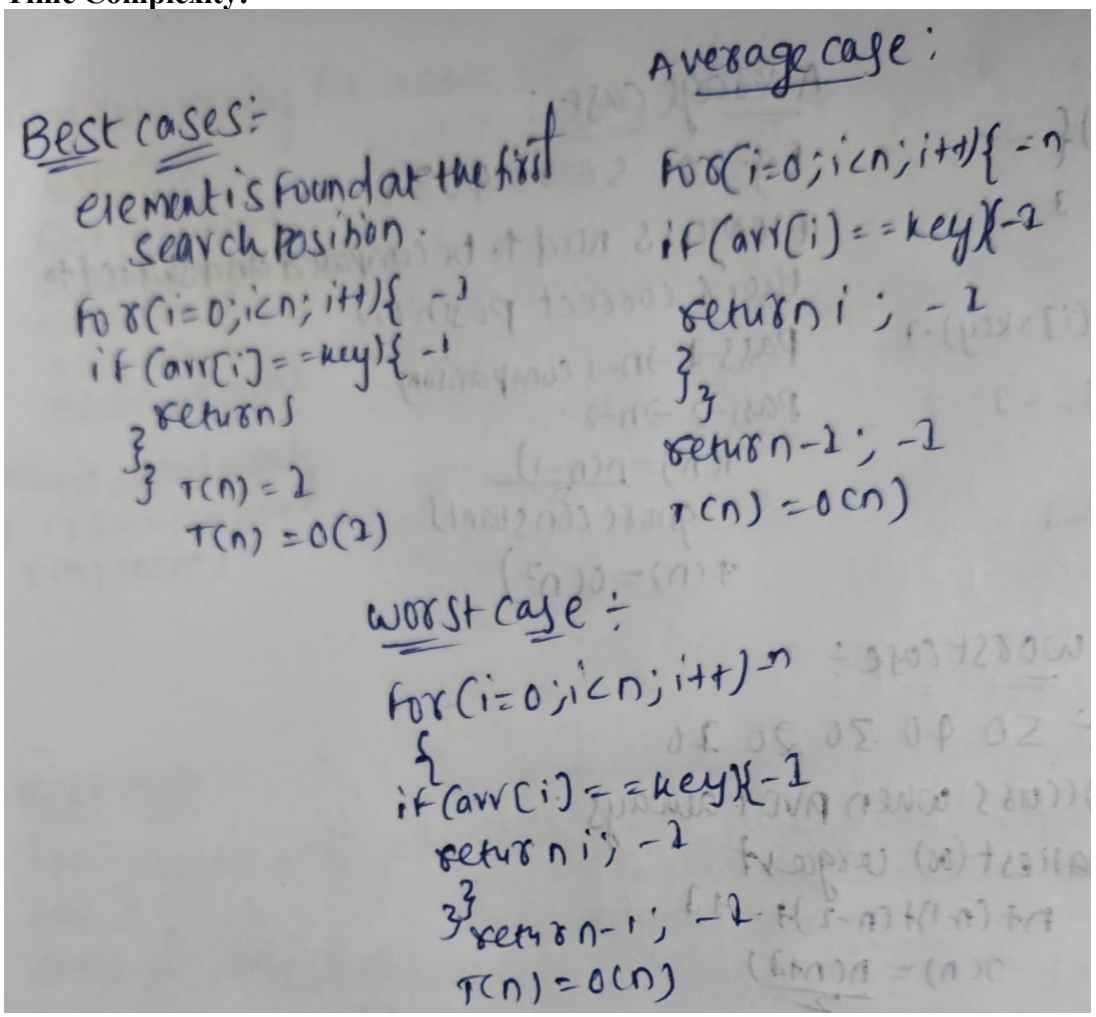
Program:

```
#include  <iostream>

using namespace std;

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    int target;
    cout << "Enter the element to search: ";
    cin >> target;

    int index = -1;
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            index = i;
            break;
        }
    }

    if (index != -1) {
        cout << "Element found at index: " << index << endl;
    } else {
        cout << "Element not found" << endl;
    }

    return 0;
}
```

Output:

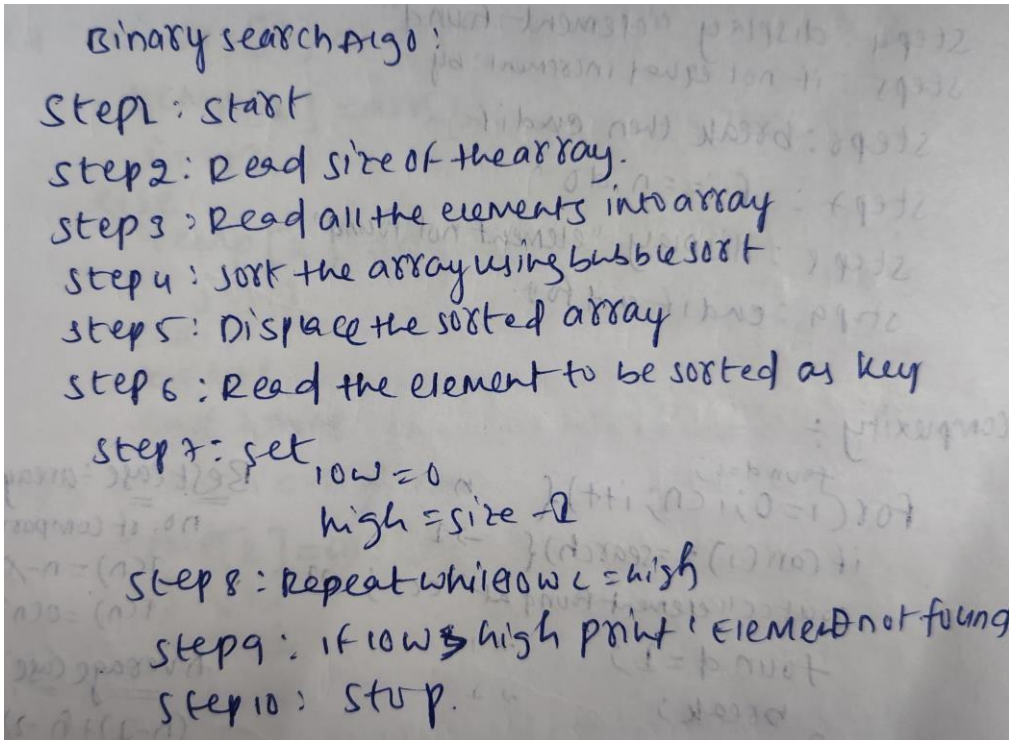
```
Enter the number of elements: 5
Enter 5 elements: 66
32
55
45
6
Enter the element to search: 6
Element found at index: 4
```

Time Complexity:



Binary search

Algorithm:

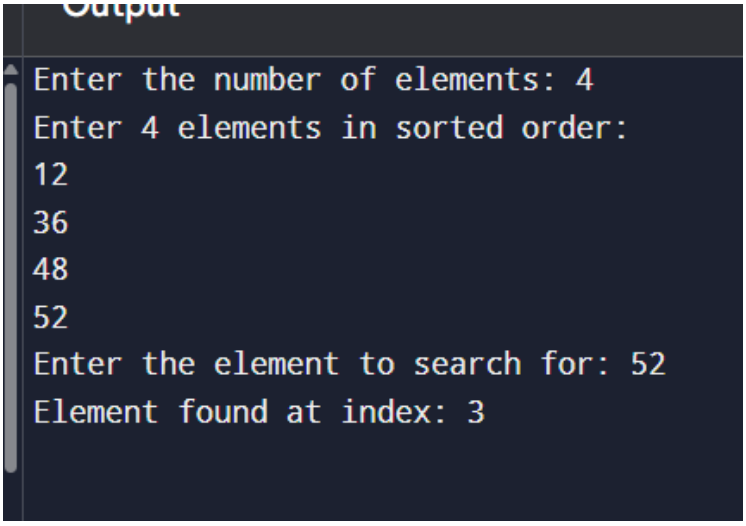


Program code:

```
#include <iostream>
#include <vector>
using namespace std;
int binarySearch(const vector<int>& arr, int target) {
    int low = 0;
    int high = arr.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }}
    return -1;
}
```

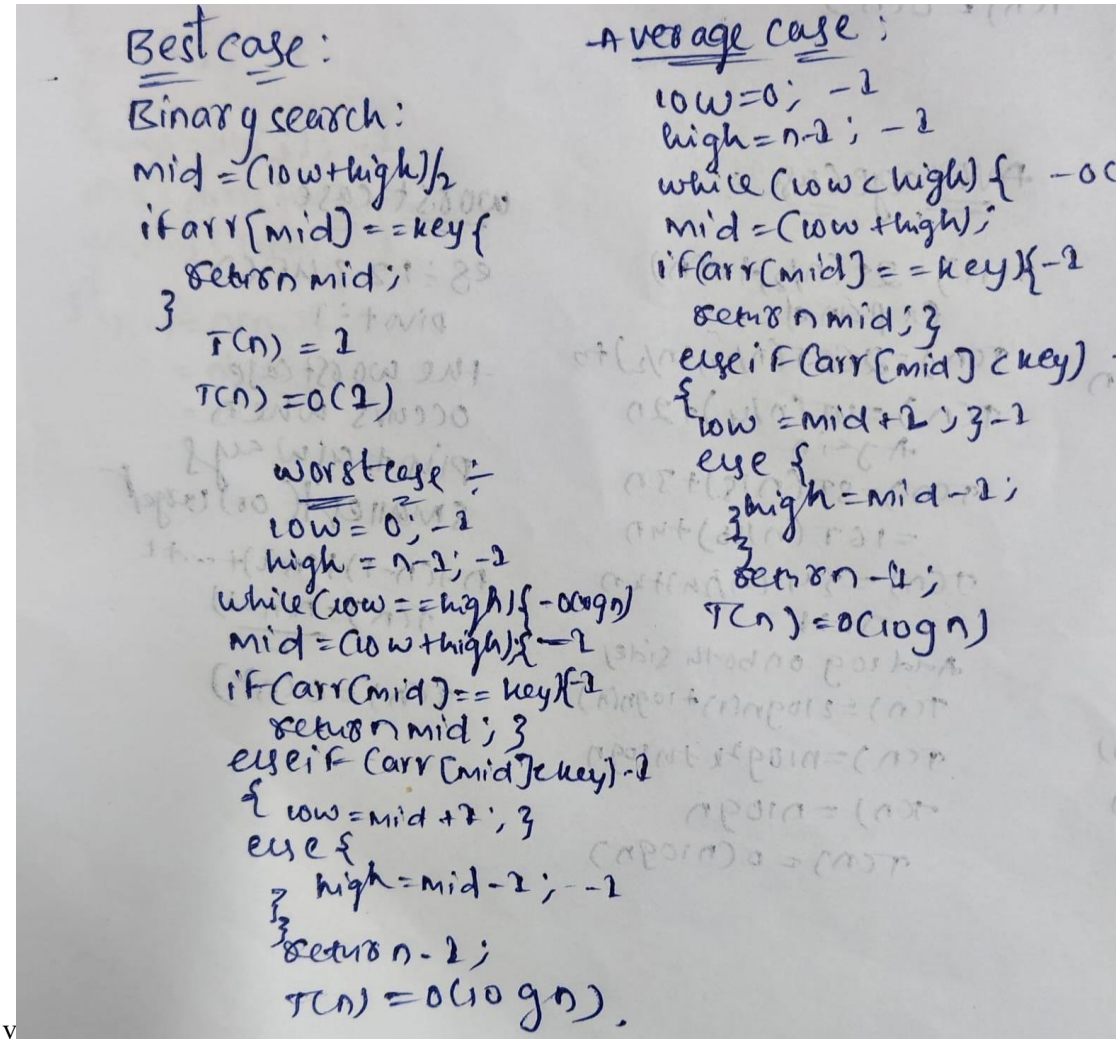
```
{
    int size;
    cout << "Enter the number of elements: ";
    cin >> size;

    vector<int> arr(size);
    cout << "Enter " << size << " elements in sorted order:\n";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }

    int target;
    cout << "Enter the element to search for: ";
    cin >> target;

    int result = binarySearch(arr, target);

    if (result != -1) {
        cout << "Element found at index: " << result << endl;
    } else {
        cout << "Element not found in the array." << endl;
    }

    return 0;
}
```

Output:



+

Time complexity:



Conclusion:

From this practical of the algorithm and programs have executed successfully. And Time complexity has categorised into three ways as Best, average and worst.